Keenan Vargas Santizo

CS 320 Software Test, Automation QA

Prof. Robert Nardelli

August 15, 2026

Before completing project one, I did not have a strong understanding of software testing techniques. In fact, I originally associated testing techniques more with the role of a tester in the SDLC than in the actual testing with the program. The testing technique I used throughout project one was JUnit testing.

For contact services, I used JUnit to test individual methods that were against the requirements. My test had assertion methods that verified newly added contacts were stored, "assertEquals("Bob", service.getContact("1").getFirstName());", and throw exceptions when a contact ID that didn't exist was trying to be deleted, "assertThrows(IllegalArgumentException.class, () -> service.deletedContact("99");});". My approach aligns to software requirements because as one source explains, "The goal of unit testing is to validate that each program unit is working as intended and meets its requirements" (Siddiq et al., 2024, pg. 314). Each program unit was validated individually as they were intended to be to ensure that the whole file was functioning correctly.

My second assignment, task services, was very similar to contact services. I also used JUnit assertion methods that verify successful and unsuccessful tests. For example, in some cases they were used to verify that retrieving an object after an update to the description produces the expected new description, "assertEquals("New Description, service.getTask("100").getTaskDescription());". In other cases, assert methods were used to throw exceptions for when a task did not exist, "assertThrows(IllegalArgumentException.class,

() -> service.updateTaskName("999", "New");});". This aligns to software requirements because testing isn't just about making everything a success, but making everything run as intended.

In appointment services, I also applied to what I have learned from the previous two assignments. For example, there is a test that verifies that adding a new appointment equals the new appointment ID, "service.addAppointment(appointment); assertEquals(appointment, service.getAppointment("001"));". One assertion method used frequently was "assertNull" that verified if deleting an appointment made the ID null, "service.deleteAppointment("001"); assertNull(service.getAppointment("001"));". As mentioned before, testing successful intended outcomes, as well as unsuccessful yet intended outcomes, is what makes this approach aligned to the software requirements. This is also what made the quality of my testing efficient and technically sound, because it tackled many elements with similar assert methods whilst being specific to that test. However, there is another reason, and it is because this was the first assignment (before project one) where the importance of coverage was mentioned and used.

"JaCoCo" was the coverage tool I used, and it is a great way to ensure your testing techniques are effective. The standard we were set was 80% or more. All the features were 80% and even appointment services were well in the 90s. By using the tool, you can see how much of your files and methods were used for testing and which areas you could add tests, and it gives you the total coverage percentage. I went back to contact services, task services, and appointment services, to apply and refine the testing so that each would have 80% coverage.

Unit testing was the only testing technique I used in project one due to being very familiar to me. Unit testing is when it takes individual units and tests them to each one of them to

verify their success. Testing techniques that I did not use that could be used here are integration and acceptance testing. Integration testing is best used when developing the frontend and the backend; this is because the tests focus on determining if multiple components in the system connect and work together in unison. Acceptance testing on the other hand can also be used in this scenario, because this project has requirements that were fulfilled and tested. While acceptance testing is mostly used more towards the end of the finished application, this can be done in a scenario where I, as the tester, am testing software towards the end of a sprint.

Testing programs require caution because testing a program on your own can be a hit or miss. In other words, you can believe that you applied every possible test for your program, but there is always something you can overlook. This actually happened to me in the appointment service assignment, I thought I covered enough testing for the assignment, but when I reviewed it for project one, it turns out I had only done 60% coverage. For this reason alone, testing can create bias that your testing is sufficient to be passible when it isn't always possible to tell. Now I know that testing software is a major responsibility that testers have to be disciplined enough to go over thoroughly. Cutting corners to rush testing may not provide any visible issues or any at all, but creating a risk assessment prevents future bugs that could potentially cause major issues like my example of the Y2K problem.

References

Siddiq, M. L., Da Silva Santos, J. C., Tanvir, R. H., Ulfat, N., Al Rifat, F., & Carvalho Lopes, V. (2024, June). Using large language models to generate junit tests: An empirical study. In *Proceedings of the 28th international conference on evaluation and assessment in software engineering* (pp. 313-322). https://dl.acm.org/doi/abs/10.1145/3661167.3661216